

# 火车订票系统设计报告

组长：张超超 学号：2023K8009906024

组员：朱首赫 学号：2023K8009906029

组员：冯曦熠 学号：2023K8009909006

## 0 分工说明

| 成员   | 负责模块                                   |
|------|----------------------------------------|
| 合作完成 | 系统架构设计                                 |
| 张超超  | 关系模式设计、核心查询功能实现、用户认证实现、订票与订单管理、管理员功能实现 |
| 朱首赫  | 代码审查与功能整合、项目结构规范化、ER 图绘制、范式细化分析、SQL 分析 |
| 冯曦熠  | 需求功能测试与验证、前端页面设计与美化、用户交互功能优化           |

## 1 总体设计说明

### 1.1 系统架构

本系统采用 Web 应用的三层结构，具体实现如下表所示：

表 1: 系统三层架构

| 层次    | 技术实现              | 说明           |
|-------|-------------------|--------------|
| 展示层   | HTML + JavaScript | 提供用户交互界面     |
| 中间层   | C++ CGI           | 处理业务逻辑，连接数据库 |
| 数据管理层 | PostgreSQL        | 存储所有业务数据     |

### 1.2 核心功能

系统实现了以下 9 个具体功能需求：

- 记录列车信息**：包括车次、站点、到发时间、票价。
- 记录座位情况**：支持座位在不同行程段内的复用。
- 乘客信息管理**：支持用户注册与登录。
- 查询车次**：支持通过具体车次号进行查询。
- 查询两地间车次**：支持直达路径及一次换乘路径的搜索。
- 查询返程信息**：基于当前查询快速切换出发与目的地。
- 预订车次座位**：实现订票业务逻辑。
- 管理订单**：支持历史订单查询与退票（取消订单）功能。
- 管理员界面**：提供全局业务数据的统计与监控。



| 列属性             | 类型          | 描述       |
|-----------------|-------------|----------|
| <u>train_id</u> | VARCHAR(10) | 车次编号（主键） |

## 4.2 运营与票务表

### 4. Train\_Station（列车经停信息表）

| 列属性            | 类型          | 描述         |
|----------------|-------------|------------|
| <u>id</u>      | SERIAL      | 记录编号（主键）   |
| train_id       | VARCHAR(10) | 车次编号（外键）   |
| station_id     | INT         | 车站编号（外键）   |
| station_order  | INT         | 该车站在车次中的顺序 |
| arrival_time   | TIME        | 到达时间       |
| departure_time | TIME        | 发车时间       |

约束：UNIQUE(train\_id, station\_order), UNIQUE(train\_id, station\_id)

### 5. Ticket\_Price（票价表）

| 列属性                 | 类型            | 描述          |
|---------------------|---------------|-------------|
| <u>train_id</u>     | VARCHAR(10)   | 车次编号（联合主键）  |
| <u>from_station</u> | INT           | 起始站编号（联合主键） |
| <u>to_station</u>   | INT           | 终点站编号（联合主键） |
| <u>seat_type</u>    | VARCHAR(20)   | 座位类型（联合主键）  |
| price               | DECIMAL(10,2) | 票价          |

### 6. Seat\_Inventory（座位库存表）

| 列属性                 | 类型          | 描述          |
|---------------------|-------------|-------------|
| <u>train_id</u>     | VARCHAR(10) | 车次编号（联合主键）  |
| <u>travel_date</u>  | DATE        | 出行日期（联合主键）  |
| <u>seat_type</u>    | VARCHAR(20) | 座位类型（联合主键）  |
| <u>from_station</u> | INT         | 起始站编号（联合主键） |
| <u>to_station</u>   | INT         | 终点站编号（联合主键） |
| remaining           | INT         | 剩余座位数       |

## 4.3 用户与交易表

### 7. User\_Info（用户表）

| 列属性            | 类型           | 描述       |
|----------------|--------------|----------|
| <u>user_id</u> | SERIAL       | 用户编号（主键） |
| username       | VARCHAR(20)  | 用户名（唯一）  |
| phone          | VARCHAR(11)  | 手机号（唯一）  |
| name           | VARCHAR(20)  | 真实姓名     |
| password       | VARCHAR(100) | 密码       |

## 8. Orders（订单表）

| 列属性             | 类型            | 描述          |
|-----------------|---------------|-------------|
| <u>order_id</u> | SERIAL        | 订单编号（主键）    |
| user_id         | INT           | 用户编号（外键）    |
| total_price     | DECIMAL(10,2) | 订单总价        |
| status          | VARCHAR(10)   | 订单状态（正常/取消） |
| create_time     | TIMESTAMP     | 下单时间        |

## 9. Order\_Item（订单明细表）

| 列属性          | 类型            | 描述       |
|--------------|---------------|----------|
| <u>id</u>    | SERIAL        | 记录编号（主键） |
| order_id     | INT           | 订单编号（外键） |
| train_id     | VARCHAR(10)   | 车次编号     |
| from_station | INT           | 起始站编号    |
| to_station   | INT           | 终点站编号    |
| seat_type    | VARCHAR(20)   | 座位类型     |
| price        | DECIMAL(10,2) | 该段票价     |
| travel_date  | DATE          | 出行日期     |

# 5 范式细化分析

本节对数据库核心表进行函数依赖（FD）分析，并基于判定条件证明其规范化程度。

## 5.1 基础地理信息：City 与 Station

City 表 (city\_id, city\_name)

- **FDs:**  $city\_id \rightarrow city\_name$ ;  $city\_name \rightarrow city\_id$  (唯一约束)。
- **候选键:**  $city\_id$  或  $city\_name$ 。
- **证明:** 决定因素均为候选键，无部分依赖或传递依赖，满足 **BCNF**。

Station 表 (station\_id, station\_name, city\_id)

- **FDs:**  $station\_id \rightarrow \{station\_name, city\_id\}$ ;  $(station\_name, city\_id) \rightarrow station\_id$ 。
- **候选键:**  $station\_id$  或  $(station\_name, city\_id)$ 。

- **证明:** 每一个行列决定因素都是候选键。虽存在  $station\_id \rightarrow city\_id$ , 但由于同一城市有多个车站, 不存在反向映射, 无传递依赖, 满足 **BCNF**。

## 5.2 运行逻辑: Train\_Station 表

模式: Train\_Station(id, train\_id, station\_id, station\_order, arrival\_time, departure\_time)

- **候选键:** id、(train\_id, station\_order)、(train\_id, station\_id)。
- **函数依赖:** 任一候选键  $\rightarrow$  所有非键属性。
- **范式证明:**
  1. 属性均不可再分, 满足 **1NF**。
  2. 候选键虽有复合形式, 但不存在非主属性对候选键的部分依赖, 满足 **2NF**。
  3. 决定因素均为候选键, 不存在非主属性间的传递依赖, 满足 **3NF** 且达到 **BCNF**。

## 5.3 定价与库存: Ticket\_Price 与 Seat\_Inventory

这两张表作为关联实体, 其属性对候选键具有 \*\* 完全函数依赖 \*\* 关系。

**Ticket\_Price** 主键为 (train\_id, from\_station, to\_station, seat\_type)。

证明: 票价 price 由车次、区间及席位类型唯一确定。任何主键的子集 (如仅车次或仅区间) 均无法决定票价, 无部分依赖, 满足 **BCNF**。

**Seat\_Inventory** 主键为 (train\_id, travel\_date, seat\_type, from\_station, to\_station)。

证明: 库存 remaining 随日期和区间动态变化。决定因素仅为主键全集, 不存在传递依赖, 满足 **BCNF**。

## 5.4 用户与订单模块

**User\_Info:** 存在 user\_id、username、phone 三个单属性候选键。由于每个决定因素都是候选键, 满足 **BCNF**。

**Orders & Order\_Item:** Orders 表通过 order\_id 唯一确定订单状态; Order\_Item 记录具体行程段。所有非键属性均直接依赖于主键, 无传递依赖, 均满足 **BCNF**。

# 6 核心功能 SQL 实现

## 6.1 需求 1: 记录列车信息

### 1.1 车次详情 (始发站、经停站、终点站及到发时间)

通过 Train\_Station 表记录列车经停信息, 包含车站顺序、到达时间、发车时间:

```
1  -- 创建列车经停信息表
2  CREATE TABLE Train_Station (
3      id SERIAL PRIMARY KEY,
4      train_id VARCHAR(10) REFERENCES Train(train_id),
5      station_id INT REFERENCES Station(station_id),
6      station_order INT NOT NULL,      -- 车站顺序
7      arrival_time TIME,              -- 到达时间
```

```
8      departure_time TIME,          -- 发车时间
9      UNIQUE(train_id, station_order),
10     UNIQUE(train_id, station_id)
11 );
```

## 1.2 站名约束 (最长 20 字符)

在 Station 表中定义站名长度约束:

```
1 CREATE TABLE Station (
2     station_id SERIAL PRIMARY KEY,
3     station_name VARCHAR(20) NOT NULL, -- 站名最长20字符
4     city_id INT NOT NULL REFERENCES City(city_id),
5     UNIQUE(station_name, city_id)
6 );
```

## 1.3 座位与票价 (二等座、一等座、商务座)

票价表支持三种座位类型, 通过 seat\_type 字段区分:

```
1 CREATE TABLE Ticket_Price (
2     train_id VARCHAR(10),
3     from_station INT,
4     to_station INT,
5     seat_type VARCHAR(20),          -- 二等座/一等座/商务座
6     price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
7     PRIMARY KEY(train_id, from_station, to_station, seat_type),
8     FOREIGN KEY(train_id, from_station)
9         REFERENCES Train_Station(train_id, station_id),
10    FOREIGN KEY(train_id, to_station)
11        REFERENCES Train_Station(train_id, station_id),
12    CHECK (from_station <> to_station)
13 );
```

## 1.4 运行假设

系统假设列车每天准时运行且票价固定不变, 票价数据在导入时一次性写入, 运行期间不修改。

# 6.2 需求 2: 记录座位情况

## 2.1 座位配额 (每类座位每天 5 个位置)

通过 ensure\_inventory() 函数在首次查询时初始化库存, 默认每段座位数为 5:

```
1 -- 座位库存初始化 (m12306_common.h 中的 ensure_inventory 函数)
2 INSERT INTO seat_inventory(train_id, travel_date, seat_type,
3                             from_station, to_station, remaining)
4 SELECT tp.train_id, $2::date, tp.seat_type,
5        tp.from_station, tp.to_station, 5    -- 默认5个座位
6 FROM ticket_price tp
7 WHERE tp.train_id=$1
8 ON CONFLICT (train_id, travel_date, seat_type,
9              from_station, to_station) DO NOTHING;
```

## 2.2 座位共享机制 (段内复用)

座位库存按区间独立管理, 同一座位可在不重叠区间重复售卖:

```
1  -- 座位库存表设计：每个区间独立记录剩余座位
2  CREATE TABLE Seat_Inventory (
3      train_id VARCHAR(10),
4      travel_date DATE,
5      seat_type VARCHAR(20),
6      from_station INT,          -- 区间起点
7      to_station INT,           -- 区间终点
8      remaining INT NOT NULL CHECK (remaining >= 0),
9      PRIMARY KEY(train_id, travel_date, seat_type,
10                  from_station, to_station),
11      ...
12 );
13 -- 例如：A-B段售出后，B-C段仍可售卖同一座位
```

## 6.3 需求 3：乘客信息管理

### 3.1 注册功能（姓名 20 字符、手机号 11 位唯一、用户名 20 字符唯一）

```
1  -- 用户表定义 (schema.sql)
2  CREATE TABLE User_Info (
3      user_id SERIAL PRIMARY KEY,
4      username VARCHAR(20) NOT NULL UNIQUE,  -- 用户名唯一
5      phone VARCHAR(11) NOT NULL UNIQUE,     -- 手机号11位唯一
6      name VARCHAR(20) NOT NULL,             -- 姓名20字符
7      password VARCHAR(100) NOT NULL
8  );
9
10 -- 注册逻辑 (register.cpp)
11 -- 1. 检查用户名是否存在
12 SELECT COUNT(*) FROM user_info WHERE username=$1;
13 -- 2. 检查手机号是否存在
14 SELECT COUNT(*) FROM user_info WHERE phone=$1;
15 -- 3. 插入新用户
16 INSERT INTO user_info(username, phone, name, password)
17 VALUES($1, $2, $3, $4);
```

### 3.2 登录功能（用户名登录，进入欢迎页面）

```
1  -- 登录验证 (login.cpp)
2  SELECT user_id, name, password FROM user_info WHERE username=$1;
3  -- 验证密码后，重定向至 home.cgi 或 admin.cgi
```

## 6.4 需求 4：查询车次

### 4.1 输入（车次号和日期）

用户通过 query\_train.html 输入车次号（如 G1）和日期。

### 4.2 显示（静态信息：各站时间、票价；动态信息：每站余票）

```
1  -- 查询车次经停站及余票 (query_train.cpp)
```

```

2 WITH start_station AS (
3     SELECT station_id AS start_sid FROM train_station
4     WHERE train_id=$1 ORDER BY station_order LIMIT 1
5 )
6 SELECT ts.station_order, c.city_name, s.station_name,
7     COALESCE(ts.arrival_time::text, '-') AS arr,
8     COALESCE(ts.departure_time::text, '-') AS dep,
9     COALESCE(tp.price::text, '-') AS fare,
10    CASE WHEN ts.station_order=1 THEN 0 ELSE COALESCE((
11        SELECT MIN(si.remaining)
12        FROM seat_inventory si
13        JOIN train_station a ON a.train_id=si.train_id
14        AND a.station_id=si.from_station
15        JOIN train_station b ON b.train_id=si.train_id
16        AND b.station_id=si.to_station
17        WHERE si.train_id=$1 AND si.travel_date=$2::date
18        AND si.seat_type=$3
19        AND a.station_order>=1 AND b.station_order=ts.station_order
20    ), 5) END AS left_seat,
21    ts.station_id
22 FROM train_station ts
23 JOIN station s ON s.station_id=ts.station_id
24 JOIN city c ON c.city_id=s.city_id
25 LEFT JOIN start_station ss ON TRUE
26 LEFT JOIN ticket_price tp ON tp.train_id=ts.train_id
27     AND tp.from_station=ss.start_sid
28     AND tp.to_station=ts.station_id AND tp.seat_type=$3
29 WHERE ts.train_id=$1
30 ORDER BY ts.station_order;

```

### 4.3 跳转（余票链接跳转订票页面）

```

1 // query_train.cpp 中生成订票链接
2 if (i > 0 && left > 0 && fare != "-") {
3     std::cout << "<a href=\"/cgi-bin/book.cgi?username=" << username
4         << "&train_id=" << train_id
5         << "&from_sid=" << start_sid
6         << "&to_sid=" << sid
7         << "&date=" << date
8         << "&seat_type=" << seat_type << ">Book</a>";
9 }

```

## 6.5 需求 5：查询两地间车次

### 5.1 输入（出发城市、到达城市、日期、时间）

用户通过 query\_route.html 输入查询条件。

### 5.2 表格 1：直达列车

```

1 -- 直达查询 (query_route.cpp)
2 SELECT tp.train_id, s1.station_id, s1.station_name,

```



```

3      s2.station_id, s2.station_name,
4      ts1.departure_time::text, ts2.arrival_time::text,
5      tp.price::text,
6      COALESCE((SELECT si.remaining FROM seat_inventory si
7                  WHERE si.train_id=tp.train_id
8                        AND si.travel_date=$3::date
9                        AND si.seat_type=$5
10                       AND si.from_station=tp.from_station
11                       AND si.to_station=tp.to_station), 5)::text AS left_seat
12 FROM ticket_price tp
13 JOIN station s1 ON s1.station_id=tp.from_station
14 JOIN city c1 ON c1.city_id=s1.city_id
15 JOIN station s2 ON s2.station_id=tp.to_station
16 JOIN city c2 ON c2.city_id=s2.city_id
17 JOIN train_station ts1 ON ts1.train_id=tp.train_id
18   AND ts1.station_id=tp.from_station
19 JOIN train_station ts2 ON ts2.train_id=tp.train_id
20   AND ts2.station_id=tp.to_station
21 WHERE c1.city_name=$1 AND c2.city_name=$2
22   AND c1.city_name<>c2.city_name      -- 过滤同一城市
23   AND tp.seat_type=$5
24   AND ts1.departure_time >= $4::time -- 时间限制
25   AND COALESCE((SELECT si.remaining ...), 5) > 0 -- 过滤无票
26 ORDER BY tp.price ASC,                -- 按票价排序
27   CASE WHEN ts2.arrival_time>=ts1.departure_time
28   THEN ts2.arrival_time-ts1.departure_time
29   ELSE ts2.arrival_time + interval '24 hour' - ts1.departure_time
30 END ASC,                               -- 按总时间排序
31 ts1.departure_time ASC                 -- 按起始时间排序
32 LIMIT 20;                             -- 最多20行

```

### 5.3 表格 2: 中转 (一次换乘)

```

1  -- 换乘查询 (query_route.cpp)
2  SELECT tp1.train_id, tp1.from_station::text, s1.station_name,
3         tp1.to_station::text, sx1.station_name, cx1.city_name,
4         tp2.train_id, tp2.from_station::text, sx2.station_name,
5         tp2.to_station::text, s2.station_name,
6         ts1.departure_time::text, tsx1.arrival_time::text,
7         tsx2.departure_time::text, ts2.arrival_time::text,
8         (tp1.price + tp2.price)::text AS total_price,
9         LEAST(
10             COALESCE((SELECT si.remaining ... WHERE ... train1 ...),5),
11             COALESCE((SELECT si.remaining ... WHERE ... train2 ...),5)
12         )::text AS left_total
13 FROM ticket_price tp1
14 JOIN ticket_price tp2 ON tp1.seat_type=$5 AND tp2.seat_type=$5
15   AND tp1.train_id<>tp2.train_id      -- 两趟不同列车
16 JOIN station s1 ON s1.station_id=tp1.from_station
17 JOIN city c1 ON c1.city_id=s1.city_id

```

```
18 JOIN station sx1 ON sx1.station_id=tp1.to_station
19 JOIN city cx1 ON cx1.city_id=sx1.city_id
20 JOIN station sx2 ON sx2.station_id=tp2.from_station
21 JOIN city cx2 ON cx2.city_id=sx2.city_id
22 JOIN station s2 ON s2.station_id=tp2.to_station
23 JOIN city c2 ON c2.city_id=s2.city_id
24 JOIN train_station ts1 ON ts1.train_id=tp1.train_id
25     AND ts1.station_id=tp1.from_station
26 JOIN train_station tsx1 ON tsx1.train_id=tp1.train_id
27     AND tsx1.station_id=tp1.to_station
28 JOIN train_station tsx2 ON tsx2.train_id=tp2.train_id
29     AND tsx2.station_id=tp2.from_station
30 JOIN train_station ts2 ON ts2.train_id=tp2.train_id
31     AND ts2.station_id=tp2.to_station
32 WHERE c1.city_name=$1 AND c2.city_name=$2
33     AND c1.city_name<>c2.city_name      -- 过滤同一城市
34     AND cx1.city_name=cx2.city_name     -- 同城换乘
35     AND ts1.departure_time >= $4::time
36     AND (
37         -- 同站换乘：等待1-4小时
38         (tp1.to_station=tp2.from_station
39             AND (tsx2.departure_time - tsx1.arrival_time)
40                 BETWEEN interval '1 hour' AND interval '4 hour')
41     OR
42         -- 异站换乘：等待2-4小时
43         (tp1.to_station<>tp2.from_station
44             AND (tsx2.departure_time - tsx1.arrival_time)
45                 BETWEEN interval '2 hour' AND interval '4 hour')
46     )
47 ORDER BY (tp1.price + tp2.price) ASC, ts1.departure_time ASC
48 LIMIT 20;
```

## 6.6 需求 6：查询返程信息

### 6.1 自动关联与预填功能

```
1 // query_route.cpp 中生成返程链接
2 std::cout << "<a href=\"/"query_route.html?username=" << username
3     << "&from_city=" << to_city      // 对调出发城市
4     << "&to_city=" << from_city      // 对调到达城市
5     << "&seat_type=" << seat_type
6     << "&date=2026-05-02"           // 默认次日
7     << "&time=00:00\">Return Trip Query</a>";
```

## 6.7 需求 7：预订车次座位

### 7.1 费用计算（票价 + 5 元订票费）

```
1 // book.cpp 中计算费用
```

```
2 double fare = 0.0; // 从 ticket_price 表获取
3 fetch_leg(conn, train_id, from_sid, to_sid, seat_type, fare, ...);
4 double service_fee = 5.0; // 每张票5元订票费
5 double total = fare + service_fee;
6
7 // 换乘订单费用计算
8 double service_fee = 10.0; // 两段票共10元订票费
9 double total = fare1 + fare2 + service_fee;
```

## 7.2 订单生成与库存更新 (事务处理)

```
1 -- 订票事务 (book.cpp)
2 BEGIN;
3
4 -- 1. 锁定并检查库存 (FOR UPDATE 悲观锁)
5 SELECT remaining FROM seat_inventory
6 WHERE train_id=$1 AND travel_date=$2::date AND seat_type=$3
7     AND from_station=$4::int AND to_station=$5::int FOR UPDATE;
8 -- 检查 remaining > 0
9
10 -- 2. 创建订单
11 INSERT INTO orders(user_id, total_price, status)
12 VALUES($1::int, $2::numeric, '正常') RETURNING order_id;
13
14 -- 3. 添加订单明细
15 INSERT INTO order_item(order_id, train_id, from_station,
16     to_station, seat_type, price, travel_date)
17 VALUES($1::int, $2, $3::int, $4::int, $5, $6::numeric, $7::date);
18
19 -- 4. 扣减库存
20 UPDATE seat_inventory SET remaining=remaining-1
21 WHERE train_id=$1 AND travel_date=$2::date AND seat_type=$3
22     AND from_station=$4::int AND to_station=$5::int;
23
24 COMMIT;
```

## 6.8 需求 8: 管理订单

### 8.1 查询历史订单 (按日期范围筛选)

```
1 -- 订单列表查询 (orders.cpp)
2 SELECT o.order_id::text, o.create_time::date::text,
3     COALESCE(sf.station_name, '-') AS from_station,
4     COALESCE(st.station_name, '-') AS to_station,
5     o.total_price::text, o.status
6 FROM orders o
7 LEFT JOIN LATERAL (
8     SELECT oi.from_station FROM order_item oi
9     WHERE oi.order_id=o.order_id ORDER BY oi.id ASC LIMIT 1
10 ) first_leg ON TRUE -- 获取第一段起点
```

```
11 LEFT JOIN LATERAL (  
12     SELECT oi.to_station FROM order_item oi  
13     WHERE oi.order_id=o.order_id ORDER BY oi.id DESC LIMIT 1  
14 ) last_leg ON TRUE      -- 获取最后一段终点  
15 LEFT JOIN station sf ON sf.station_id=first_leg.from_station  
16 LEFT JOIN station st ON st.station_id=last_leg.to_station  
17 WHERE o.user_id=$1::int  
18     AND o.create_time::date BETWEEN $2::date AND $3::date  
19 ORDER BY o.order_id DESC;
```

## 8.2 订单详情查询

```
1  -- 订单明细 (orders.cpp)  
2  SELECT oi.id::text, oi.train_id, sf.station_name, st.station_name,  
3         oi.seat_type, oi.price::text, oi.travel_date::text,  
4         COALESCE(ts1.departure_time::text, '-') AS depart_time,  
5         COALESCE(ts2.arrival_time::text, '-') AS arrive_time  
6  FROM order_item oi  
7  JOIN orders o ON o.order_id=oi.order_id  
8  JOIN station sf ON sf.station_id=oi.from_station  
9  JOIN station st ON st.station_id=oi.to_station  
10 LEFT JOIN train_station ts1 ON ts1.train_id=oi.train_id  
11     AND ts1.station_id=oi.from_station  
12 LEFT JOIN train_station ts2 ON ts2.train_id=oi.train_id  
13     AND ts2.station_id=oi.to_station  
14 WHERE oi.order_id=$1::int AND o.user_id=$2::int  
15 ORDER BY oi.id;
```

## 8.3 退票 (取消订单, 恢复库存)

```
1  -- 退票事务 (orders.cpp)  
2  BEGIN;  
3  
4  -- 1. 锁定订单  
5  SELECT status FROM orders  
6  WHERE order_id=$1::int AND user_id=$2::int FOR UPDATE;  
7  -- 检查状态为'正常'  
8  
9  -- 2. 获取订单明细并恢复库存  
10 SELECT train_id, seat_type, from_station, to_station, travel_date::text  
11 FROM order_item WHERE order_id=$1::int;  
12  
13 -- 对每段行程恢复库存  
14 UPDATE seat_inventory SET remaining=remaining+1  
15 WHERE train_id=$1 AND seat_type=$2  
16     AND from_station=$3::int AND to_station=$4::int  
17     AND travel_date=$5::date;  
18  
19 -- 3. 更新订单状态  
20 UPDATE orders SET status='取消'  
21 WHERE order_id=$1::int AND user_id=$2::int;
```

```
22  
23 COMMIT;
```

## 6.9 需求 9: 管理员界面

### 9.1 固定 Admin 账户

```
1 // login.cpp 中确保 admin 用户存在  
2 INSERT INTO user_info(username, phone, name, password)  
3 VALUES('admin', '0000000000', 'Admin', 'admin')  
4 ON CONFLICT (username) DO NOTHING;  
5  
6 // m12306_common.h 中判断管理员  
7 inline bool is_admin(const std::string &username) {  
8     return username == "admin";  
9 }
```

### 9.2 统计看板

```
1 -- 总订单数 (admin.cpp)  
2 SELECT COUNT(*)::text FROM orders WHERE status='正常';  
3  
4 -- 总成交金额 (admin.cpp)  
5 SELECT COALESCE(SUM(total_price),0)::text FROM orders  
6 WHERE status='正常';  
7  
8 -- 热点车次排行前10 (admin.cpp)  
9 SELECT oi.train_id, COUNT(*)::text cnt  
10 FROM order_item oi  
11 JOIN orders o ON o.order_id=oi.order_id  
12 WHERE o.status='正常'  
13 GROUP BY oi.train_id  
14 ORDER BY cnt::int DESC LIMIT 10;  
15  
16 -- 所有注册用户列表 (admin.cpp)  
17 SELECT username, name, phone FROM user_info ORDER BY user_id;  
18  
19 -- 查看指定用户订单 (admin.cpp)  
20 SELECT o.order_id::text, o.status, o.total_price::text,  
21     o.create_time::date::text  
22 FROM orders o  
23 JOIN user_info u ON u.user_id=o.user_id  
24 WHERE u.username=$1  
25 ORDER BY o.order_id DESC;
```